CSA1735 – Artificial Intelligence

CO1 Assessment 1 (AT1)

Student: Thirulogasundar Y | **Register No:** 192321073

Slot A | Questions: 3, 4, 10, 12, 15

Q3. Exam Timetabling (Constraint Satisfaction Problem)

CSP formulation

- **Variables:** one per exam, X = {M (Maths), P (Physics), C (Chemistry), S (CS), E (English)}.
- **Domains:** time slots D = {1, 2, 3} for every exam.
- **Constraints:** two exams that share at least one student must not be in the same slot. Assumed conflicts: M≠P, M≠S, P≠C, C≠S, S≠E. Further hard constraints can be added, such as room capacity and invigilator availability; soft constraints include minimising back-to-back exams for a student.
- **Goal:** a complete assignment that satisfies all constraints.

Constraint graph: nodes are exams and edges are conflicts (M–P, M–S, P–C, C–S, S–E). This is graph colouring where slots are colours.

Solving by backtracking with MRV, degree heuristic and forward checking

Step	Choice	Reason	Domains after forward checking
1	S = 1	S has the highest degree (3)	M{2,3}, C{2,3}, E{2,3}, P{1,2,3}
2	M = 2	M has the smallest domain (MRV)	P{1,3}, C{2,3}, E{2,3}
3	C = 2	smallest domain; consistent with M, S	P{1,3}, E{2,3}
4	P = 1	consistent with M=2 and C=2	E{2,3}
5	E = 2	consistent with S=1	–

Solution: S = 1, M = 2, C = 2, P = 1, E = 2. Check: M≠P (2,1), M≠S (2,1), P≠C (1,2), C≠S (2,1), S≠E (1,2) → all satisfied. Only 2 slots are needed (chromatic number 2).

Notes: If a domain becomes empty, the algorithm backtracks. Arc consistency (AC-3) can prune domains before and during search. Min-conflicts local search is a good alternative for large timetables.

Q4. Route Finding with A* Search

Evaluation function: f(n) = g(n) + h(n), where g(n) is the path cost from the start and h(n) is the estimated cost to the goal. h must be admissible (never overestimates) and, for graph search, consistent.

Assumed map (edge costs): S–A = 1, S–B = 4, A–B = 2, A–C = 5, A–G = 12, B–C = 2, C–G = 3

Heuristic (straight-line estimate): h(S)=7, h(A)=6, h(B)=4, h(C)=2, h(G)=0

Step	Expand	Generated (g, f)	Open list after step
1	S (f=7)	A (1,7), B (4,8)	A7, B8
2	A (f=7)	B via A (3,7) improves 4→3; C (6,8); G (13,13)	B7, C8, G13
3	B (f=7)	C via B (5,7) improves 6→5	C7, G13
4	C (f=7)	G via C (8,8) improves 13→8	G8
5	G (f=8)	Goal reached	–

Optimal path: S → A → B → C → G, **cost = 8**. Nodes expanded: S, A, B, C, G.

Properties: Complete and optimal when h is admissible (tree search) or consistent (graph search). Time and space are exponential in the worst case, O(b^d). A* is optimally efficient: no other optimal algorithm using the same h expands fewer nodes.

Q10. Problem Formulation: Tower of Hanoi

Problem: three pegs (A, B, C) and n disks of different sizes stacked on A with the largest at the bottom. Move all disks to C, moving one disk at a time, never placing a larger disk on a smaller one.

Formulation

- **States:** an assignment of every disk to a peg, with disks on each peg ordered by size (larger below smaller). Represent as (peg of disk n, …, peg of disk 1). There are **3^n** legal states.
- **Initial state:** all disks on peg A: (A, A, …, A).
- **Actions:** move the top disk of one peg to another peg.
- **Legal-move condition:** the source peg is not empty, and the destination is empty or its top disk is larger than the moved disk.
- **Transition model:** the state with that disk's peg changed.
- **Goal test:** all disks on peg C.
- **Path cost:** 1 per move.

Optimal solution

The minimum number of moves is **2^n − 1** (n = 3 gives 7 moves; n = 4 gives 15).

Recursive solution: Hanoi(n, from, to, via): if n = 0 return; Hanoi(n−1, from, via, to); move disk n from from to to; Hanoi(n−1, via, to, from).

Example, 3 disks (D1 smallest):

Move	Action	State (D3, D2, D1)
0	Start	(A, A, A)
1	D1: A → C	(A, A, C)
2	D2: A → B	(A, B, C)

Move	Action	State (D3, D2, D1)
3	D1: C → B	(A, B, B)
4	D3: A → C	(C, B, B)
5	D1: B → A	(C, B, A)
6	D2: B → C	(C, C, A)
7	D1: A → C	(C, C, C) goal

Search notes: the state space (a Sierpinski-triangle shaped graph) has branching factor of at most 3. BFS or A* find the optimal 7-move path; DFS must check repeated states because moves can be undone.

Q12. Mars Rover Agent

Percepts

Camera images, stereo/depth data, LIDAR/obstacle distance, wheel odometry, IMU (tilt, orientation), GPS/star-tracker position, temperature and radiation sensors, battery level, soil/rock spectrometer readings, radio commands from Earth.

Environment

Mars surface: rocky, sandy, uneven terrain, dust storms, extreme temperature, communication delay with Earth.

Property	Type	Reason
Observability	Partially observable	Sensors have limited range; terrain beyond view is unknown

Property	Type	Reason
Agents	Single agent	Only the rover acts (Earth control is a supervisor)
Determinism	Stochastic	Wheel slip, dust and sensor noise give uncertain outcomes
Episodic/Sequential	Sequential	Current moves affect later safety and energy
Static/Dynamic	Dynamic	Weather, lighting and dust change while it deliberates
Discrete/Continuous	Continuous	Position, speed and time are continuous
Known/Unknown	Partly unknown	Terrain must be learned

Actions (actuators)

Move forward/backward, turn, stop, climb or avoid obstacles, drill, scoop soil, move robotic arm, take photos, run experiments, deploy/orient solar panels, transmit data, enter sleep/safe mode.

Performance measure

Number and quality of scientific samples/data collected, area explored, mission success, safety (no tip-over, no collisions), energy efficiency, successful data transmission, time taken.

Agent architecture

A **utility-based, model-based agent with a layered architecture**:

1. **Sensors → state update:** the internal model of terrain, position and battery is updated from percepts.
2. **Reactive layer:** immediate reflexes (stop when tilt is too high, avoid an obstacle, enter safe mode on low battery).
3. **Planning layer:** path planning (A*), task scheduling for science targets.
4. **Utility function:** trades off science value, risk and energy.
5. **Actuators:** execute the chosen action; a **learning element** can improve terrain classification over time.

Q15. Logistics Delivery Network (Uniform Cost Search)

Uniform Cost Search (UCS) expands the frontier node with the lowest path cost g(n), using a priority queue. It is Dijkstra's algorithm applied to a search tree or graph.

Assumed network (road distance in km): Hub H–A = 4, H–B = 2, B–A = 1, A–C = 5, B–C = 8, B–D = 10, C–G = 6, D–G = 3. Find the cheapest route from the hub H to the destination G.

Formulation

- **States:** locations in the network. **Initial state:** H. **Goal:** G.
- **Actions:** travel along a road to an adjacent location. **Path cost:** sum of road distances.

Trace

Step	Node expanded (g)	Frontier after expansion (node: g)
1	H (0)	B: 2, A: 4
2	B (2)	A: 3 (improved from 4), C: 10, D: 12
3	A (3)	C: 8 (improved from 10), D: 12
4	C (8)	D: 12, G: 14
5	D (12)	G stays 14 (via D would be 15)
6	G (14)	Goal selected for expansion, so the search stops

Optimal route: H → B → A → C → G, **total cost = 14 km** (2 + 1 + 5 + 6). The goal test is applied when a node is selected for expansion, not when it is generated, otherwise the first-found path (cost 15 via D) could be wrongly returned.

Pseudocode

```
function UCS(problem):
  frontier ← priority queue ordered by g, containing (start, g=0)
  explored ← empty set
  loop:
    if frontier is empty: return failure
    node ← pop lowest-g node from frontier
```

```
  if GOAL-TEST(node.state): return path to node
  add node.state to explored
  for each neighbour n of node.state:
    g' ← node.g + cost(node.state, n)
    if n not in explored and not in frontier: insert (n, g')
    else if n in frontier with higher g: replace it with (n, g')
```

Properties: complete if every step cost is at least some $\varepsilon > 0$; **optimal**; time and space $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$, where C^* is the optimal cost. With equal step costs UCS behaves like BFS. UCS expands in all directions (no heuristic), so A* with a good heuristic is faster.